FIT3155
Advanced algorithms and data structures

Week 2 lecture notes by Taylor Kearney
Based on lecture slides by Arun Konagurthu

MONASH University

Faculty of Information Technology

September 11, 2025

Contents

1 The Boyer-Moore and Knuth-Morris-Pratt algorithms . . . 2
1.1 The Boyer-Moore algorithm . . . 2
1.1.1 Right-to-left scanning . . . 2
1.1.2 The bad character rule . . . 3
1.1.3 The extended bad character rule . . . 6
1.1.4 Good suffix rule . . . 9
1.1.5 Galil's optimisation . . . 14
1.1.6 Summary . . . 17
2 The Knuth-Morris-Pratt (KMP) algorithm . . . 18
2.1 The algorithm . . . 18

1 The Boyer-Moore and Knuth-Morris-Pratt algorithms

Let's now return to the ideas discussed in week 1 for improving on naive pattern matching. We claimed that by preprocessing the pattern a more intelligent algorithm could avoid unnecessary comparisons and alignments between the pattern and the text. The avoidance of these unnecessary comparisons would then yield a linear time solution to the exact pattern matching problem. Although the linear time solution we discussed in the previous section can be seen to preprocess the pattern[1], the actual matching is performed by calculating the Z-values of the text. This method certainly avoids unnecessary comparisons by using the previously computed Z-values, but it never skips over sections of the text where matches cannot occur and is quite different in spirit from the optimisations we initially proposed. In this section we will resolve this discrepancy by examining two algorithms - the Boyer-Moore algorithm and the Knuth-Morris-Pratt (KMP) algorithm - that take this more intuitive approach to the exact pattern matching problem.

Both the Boyer-Moore and KMP algorithms are linear time methods and have a worst case complexity of $O(m + n)$, assuming a pattern and text of length m and n respectively. This can make their examination seem somewhat academic since we have already described a linear time approach based on the Z-algorithm, which is arguably much simpler than either of Boyer-Moore or KMP. However, both Boyer-Moore and KMP can be seen to hold several distinct advantages over a Z-algorithm based approach. In particular, we will see that although the worst case of Boyer-Moore is linear, on average it runs in *sublinear* time making it the algorithm of choice in many applications. In contrast KMP holds little advantage over the Z-algorithm based approach in terms of performance, but can be easily generalised to more complex pattern matching problems that are not amenable to Z-algorithm methods. Since Boyer-Moore and KMP are very similar in spirit we will dedicate the majority of this section to Boyer-Moore as it is of greater practical importance and is typically the more challenging of the two methods to grasp. We will then conclude with a brief discussion of KMP illuminating many of its inner workings by noting its similarities to Boyer-Moore.

1.1 The Boyer-Moore algorithm

Similar to the naive method, Boyer-Moore successively aligns the pattern with the text and checks for a complete match in each alignment. After a mismatch, or a complete match, the pattern is shifted to the right with respect to the text generating a new alignment. To improve on the naive method the algorithm incorporates three clever ideas:

- right-to-left scanning,
- the bad character shift rule,
- the good suffix shift rule,

each of which we explore below.

1.1.1 Right-to-left scanning

For a given alignment between the pattern and the text, naive pattern matching will compare the characters of the pattern to those in text in a left-to-right scan. That is, in an alignment of $\mathbf{pat}[1 \dots m]$ with $\mathbf{txt}[j \dots j + m - 1]$, we begin by comparing $\mathbf{pat}[1]$ and $\mathbf{txt}[j]$, followed by $\mathbf{pat}[2]$ and $\mathbf{txt}[j + 1]$ and so on moving through the pattern and text in a left-to-right scan. In contrast the Boyer-Moore algorithm scans for matched characters right to left.

Example 1: Right-to-left scanning

Let's consider the alignment shown below:

```
      1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
txt:  x p b c t b x a b a b x c t b p q

            1 2 3 4 5 6 7
pat:        t p a b x a b
```

Scanning from right to left we would make the following comparisons:

- Compare $\mathbf{pat}[7] \equiv \mathtt{b}$ with $\mathbf{txt}[9] \equiv \mathtt{b}$: match.

[1] You should convince yourself that Z-algorithm based patter matching does indeed need to preprocess the pattern before it can begin searching for matches in the text.

- Compare **pat**[6] ≡ a with **txt**[8] ≡ a: match.
- Compare **pat**[5] ≡ x with **txt**[7] ≡ x: match.
- Compare **pat**[4] ≡ b with **txt**[6] ≡ b: match.
- Compare **pat**[3] ≡ a with **txt**[5] ≡ t: mismatch.

Obviously this 'backwards' scanning alone offers no performance benefits in comparison to the left-to-right scan utilised by the naive method. However, the direction of our scan is not inconsequential and we will return to its importance after discussing the two shift rules that it enables.

1.1.2 The bad character rule

Example 2: A simple bad character shift

If we return to example 1, we find a mismatch when comparing **pat**[3] ≡ a with **txt**[5] ≡ t. We now wish to shift the pattern to the right with respect to the text, but would like to shift further than the naive shift of 1 position. By examining the pattern we can see that the **rightmost** occurrence of the *bad character* i.e. the mismatched character in the text (**txt**[5] ≡ t), occurs at position 1 of the pattern (**pat**[1] ≡ t). As this is the rightmost occurrence of the mismatched character ('t') we can safely shift the pattern so as to align **pat**[1] ≡ t with **txt**[5] ≡ t in the next alignment. In this case the *bad character rule* would tell us to shift the pattern 2 places to the right as shown below.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
txt:	x	p	b	c	t	b	x	a	b	a	b	x	c	t	b	p	q
pat:			t	p	a	b	x	a	b								
					✗	✓	✓	✓	✓								
pat:					t	p	a	b	x	a	b						

We will now formalise this idea so that we can realise it algorithmically. Recall that we only wish to preprocess the pattern, and so we want to store the **rightmost** occurrences of each character that appears in the pattern. Let's define,

$$R(x) = \begin{cases} \text{Position of the rightmost occurrence of } x \text{ in the pattern} & \text{if } x \text{ occurs in pattern,} \\ 0 & \text{otherwise.} \end{cases}$$

For instance, in the previous example we would have: $R(t) = 1$, $R(b) = 7$, and $R(c) = 0$, as 'c' only appears in the text and not the pattern. All that remains is to determine how to use the $R(x)$ values to calculate how many places we should shift by when a mismatch occurs.

Let's consider some general iteration where **txt**$[j \dots j+m-1]$ and **pat**$[1 \dots m]$ are being compared via right-to-left scan, and a mismatch occurs at position k in the pattern, i.e.

$$\mathbf{x} \equiv \mathbf{txt}[j+k-1] \neq \mathbf{pat}[k] \equiv \mathbf{y}$$

as shown in figure 1.

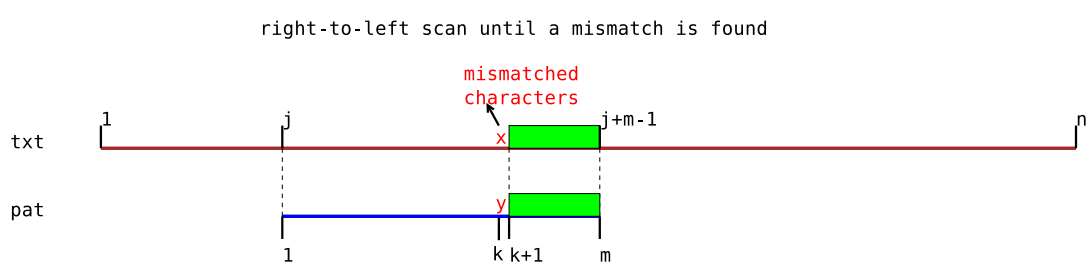


Figure 1: A general iteration of Boyer Moore where **txt**$[j \dots j+m-1]$ and **pat**$[1 \dots m]$ are being compared via right-to-left scan and a mismatch occurs at position k in the pattern.

In this case we want to align the rightmost occurrence of x in the pattern with **txt**$[j+k-1]$. Assuming that $R(x) < k$ (see figure 2) we need to shift the pattern $k - R(x)$ places to the right, as shown in figure 3.

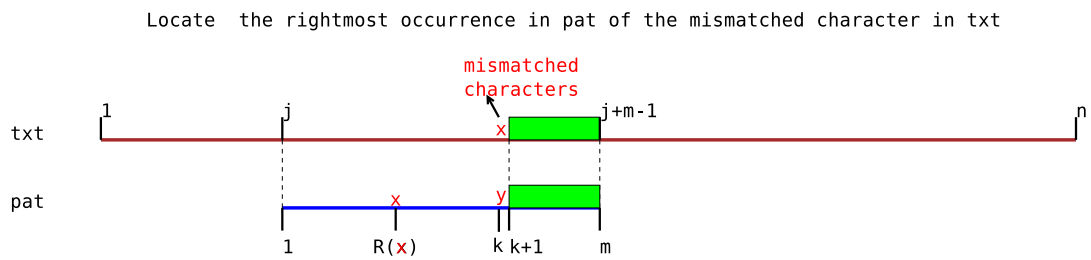


Figure 2: The case where the position of the rightmost occurrence of the bad character ('x') in the pattern is to the left of the mismatch point k, i.e. $R(x) < k$.

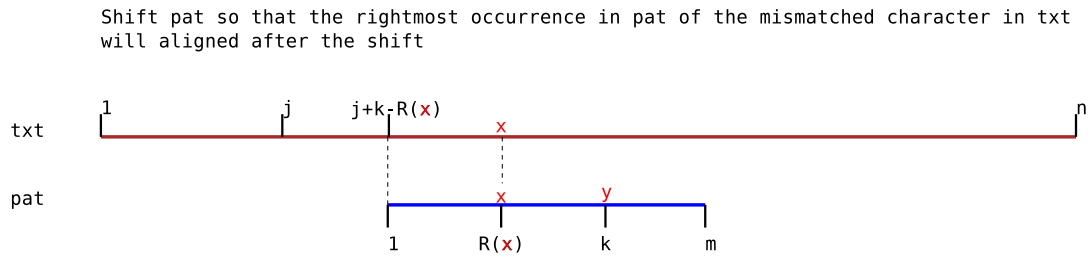


Figure 3: Assuming that $R(x) < k$, shifting the pattern to the right by $k - R(x)$ places will align the character at position $R(x)$ with the bad character (x) in the text.

If $R(x) = 0$ then the character x does not appear in the pattern and we can never find a complete match when **any** character of the pattern is aligned with x. In this case it is safe to shift the whole pattern past x ($\mathbf{txt}[j+k-1]$), as shown in figure 4. To do so we need to shift the pattern k places to the right or equivalently by $k - R(x)$ places as $R(x) = 0$. This gives our definition of $R(x)$ operational significance as we do not need to alter the general shift rule to account for this case. Conveniently, this definition is also of conceptual significance as we are 1 indexing, and so position 0 does not exist in the pattern.

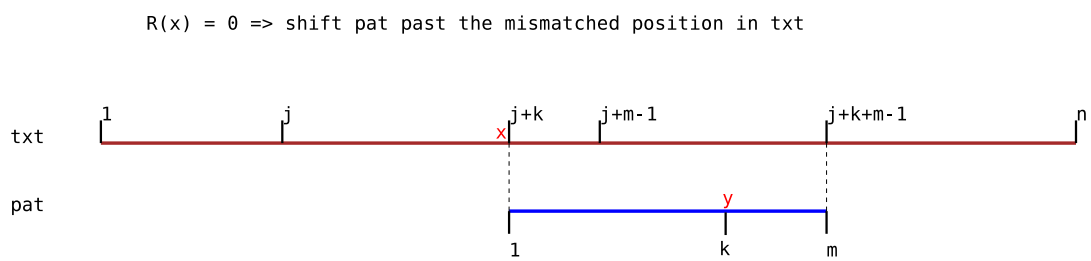


Figure 4: If the mismatched character in the text x doesn't occur in the pattern, then $R(x) = 0$ and it is safe to shift the whole pattern past the occurrence of x in the text.

But, what should we do if $R(x) > k$ as shown in figure 5? Our general shift rule would tells us to shift the pattern right by $k - R(x)$ places, but $k - R(x) < 0$ in this case! Obviously the bad character rule can't be used to help inform our shift here, and we must resort to shifting naively shifting the pattern by a single position to the right.

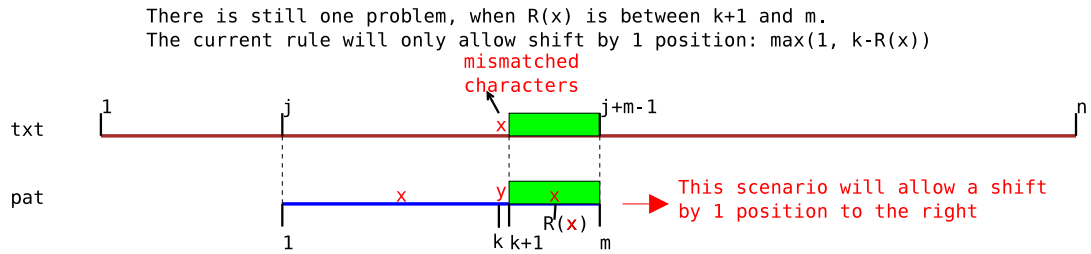


Figure 5: If the rightmost occurrence of the mismatched character in the text (x), is to the right of the mismatched position (k) in the pattern, i.e. $R(x) > k$, then shift the pattern to the right by 1 place.

Example 3: The bad character rule

Let:

$\mathbf{txt}[1\ldots17] =$ 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
x p b c t b x a b a c b x t b p q a and $\mathbf{pat}[1\ldots7] =$ 1 2 3 4 5 6 7
t b a p x a b .

The preprocessing of the pattern to compute the $R(x)$ values would yield:

x	a	b	c	p	q	t	x
$R(x)$	6	7	0	4	0	1	5

These values would then be used to shift the pattern as follows:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt:	x	p	b	c	t	b	x	a	b	a	c	b	x	t	b	p	q	a
pat:	t	b	a	p	x	a	b											
iteration 1							✗											
pat:			t	b	a	p	x	a	b									
iteration 2						✗	✓	✓	✓									
pat:				t	b	a	p	x	a	b								
iteration 3										✗								
pat:					t	b	a	p	x	a	b							
iteration 4											✗							
pat:												t	b	a	p	x	a	b
iteration 5																		✗

Bad character rule summary

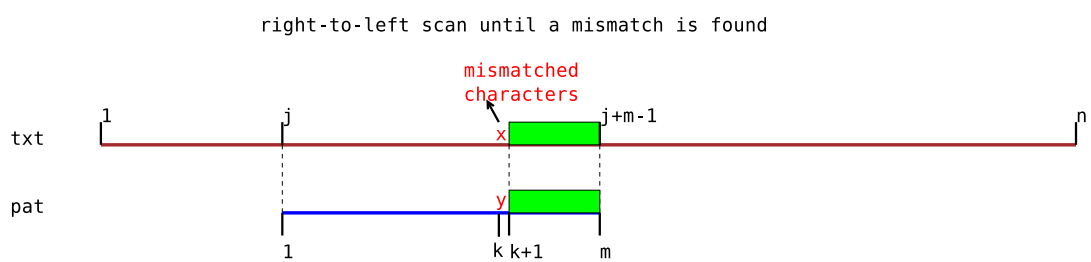
Let the pattern and the text be defined over some alphabet $\aleph$.

- Calculate,

$$R(x) = \begin{cases} \text{Position of the rightmost occurrence of } x \text{ in the pattern} & \text{if } x \text{ occurs in pattern,} \\ 0 & \text{otherwise,} \end{cases}$$

  for each character $x \in \aleph$, by preprocessing the pattern. This can be done in $O(m)$-time and $O(|\aleph|)$-space.

- Let x denote the character $\mathbf{txt}[j+k-1]$. If a mismatch occurs at position k in the pattern during an iteration where $\mathbf{txt}[j \ldots j+m-1]$ and $\mathbf{pat}[1 \ldots m]$ are being compared via right-to-left scan (shown below), then it is safe to shift the pattern to the right by, $\mathbf{max}(k - R(x), 1)$ places.



Question 1: Bad character preprocessing

Describe using pseudocode, or otherwise, how to perform the preprocessing required for the bad character rule.

1.1.3 The extended bad character rule

The bad character rule is most effective when mismatches occur towards the right end of the pattern, but reduces to naive shifting when the bad character occurs to the right of the mismatch point in the pattern. This situation becomes increasingly common as we reduce the size of our alphabet, or when the text contains many substrings that almost, but don't exactly match the pattern. To achieve better performance in these cases we would like to extend the preprocessing of the pattern to enable a more robust shift rule.

Consider iteration 2 in example 3. Here $\mathbf{pat}[4] \equiv p$ mismatches with $\mathbf{txt}[6] \equiv b$, but we only shift the pattern by 1 place as $R(b) = 7$. However, we cannot obtain a complete match until $\mathbf{pat}[2] \equiv b$ is aligned with $\mathbf{txt}[6] \equiv b$, and so we could safely shift the pattern by 2 places instead. In general if the character at position k in the pattern mismatches with $\mathbf{txt}[j+k-1] \equiv x$ in the text, we want to align the rightmost occurrence of x that is **to the left** of k in the pattern with x as shown in figure 6. This will give us the best possible bad character shift and we define the *extended bad character rule* by simply replacing $R(x)$ with $R_k(x)$ which is defined as,

$$R_k(x) = \begin{cases} \text{Position of the rightmost occurrence of } x \text{ to the left of } k \text{ in the pattern} & \text{if } x \text{ occurs to the left of } k \text{ in the pattern,} \\ 0 & \text{otherwise.} \end{cases}$$

Furthermore, we no longer need to treat the $R(x) > k$ case separately. Shifting by $k - R_k(x)$ will always give a well defined shift since $R_k(x)$ always refers to a character to the left of k. If no such character exists the extended bad character rule instructs us to shift the entire pattern past the mismatched position in the text.

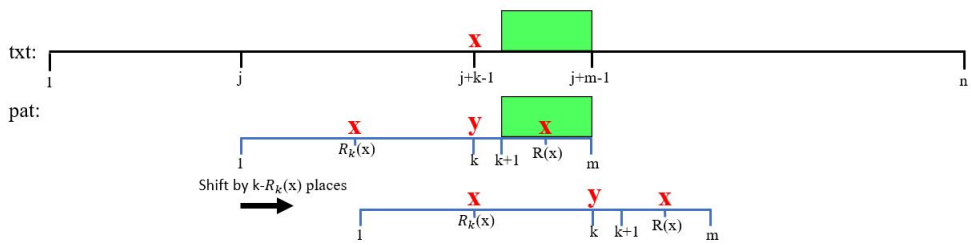


Figure 6: If the mismatched character (x) in the text appears to the left of the mismatch point k in the pattern, then aligning the rightmost such occurrence in the pattern with the mismatched x in the text, is the optimal bad character shift.

Example 4: Extended bad character preprocessing

If $\mathbf{pat}[1\ldots7] = \overset{1}{\texttt{t}}\ \overset{2}{\texttt{b}}\ \overset{3}{\texttt{a}}\ \overset{4}{\texttt{p}}\ \overset{5}{\texttt{x}}\ \overset{6}{\texttt{a}}\ \overset{7}{\texttt{b}}$, then we could store the corresponding $R_k(x)$ values in the following 2D array:

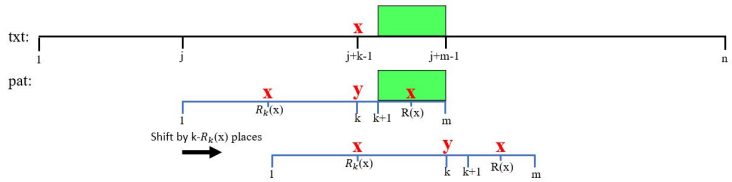
x	a	b	c	p	q	t	x
$R_1(x)$	0	0	0	0	0	0	0
$R_2(x)$	0	0	0	0	0	1	0
$R_3(x)$	0	2	0	0	0	1	0
$R_4(x)$	3	2	0	0	0	1	0
$R_5(x)$	3	2	0	4	0	1	0
$R_6(x)$	3	2	0	4	0	1	5
$R_7(x)$	6	2	0	4	0	1	5

We should note that every entry in the first row is 0 as if we mismatch on the first character in the pattern i.e. $\mathbf{pat}[1] \equiv t$, then we simply shift past the mismatched character in the text. Furthermore, some of the entries will never be used. For instance $R_7(b) = 2$, but to use this value we would need $\mathbf{pat}[7] \equiv b$ to mismatch with a 'b' in the text which is impossible.

Extended bad character rule summary

Let the pattern and the text be defined over some alphabet $\aleph$.

- Calculate $R_k(x)$ for each character $x \in \aleph$, by preprocessing the pattern. This can be done in $O(|\aleph|m)$ time and space by constructing a 2D array to store the $R_k(x)$ values.
- Let x denote the character $\mathbf{txt}[j+k-1]$. If a mismatch occurs at position k in the pattern during an iteration where $\mathbf{txt}[j\ldots j+m-1]$ and $\mathbf{pat}[1\ldots m]$ are being compared via right-to-left scan (shown below), then it is safe to shift the pattern to the right by, $k - R_k(x)$ places.



Question 2: Extended bad character preprocessing

1. Describe using pseudocode, or otherwise, how to perform the preprocessing required for the extended bad character rule. Begin with the 2D array approach suggested above. *Hint: you should use dynamic programming.*
2. Explain how you could store the $R_k(x)$ values in a more space efficient manner and discuss how this would impact the time complexity of the search phase. *Hint: do we need to explicitly store the whole 2D array?*

Question 3: Bad character implementation issues

At first glance it would appear that our extended bad character rule elegantly handles all of the cases that might arise when performing exact pattern matching, but what happens if we find an occurrence of the pattern in the text? What would be the bad character in this case, and how many places is it safe to shift the pattern by? *Note: This is not an issue in Boyer-Moore as it is handled by the good suffix rule.*

1.1.4 Good suffix rule

The (extended) bad character rule is a significant improvement over naive pattern matching and performs particularly well for natural languages, such as English, due to their sizable alphabets, and the variety of characters that appear within words. However, the bad character rule alone does not guarantee linear time pattern matching.

Example 5: The worst case of the bad character rule

Let:

$$\mathbf{txt}[1\ldots10] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{a}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{a}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\mathtt{a}}\ \overset{8}{\mathtt{a}}\ \overset{9}{\mathtt{a}}\ \overset{10}{\mathtt{a}} \quad \text{and } \mathbf{pat}[1\ldots3] = \overset{1}{\mathtt{b}}\ \overset{2}{\mathtt{a}}\ \overset{3}{\mathtt{a}}\ .$$

In this case when scanning right-to-left every alignment between **pat** and **txt** will yield two matches followed by a mismatch. The mismatch always occurs at position 1 of the pattern and so both the extended and regular bad character rule would tell us to shift by a single position each iteration, giving $(10-3+1)\times 3 = 24$ comparisons overall. Evidently this reduces to naive pattern matching with some extra preprocessing!

Example 5 illustrates the worst case of the bad character rule and demonstrates that it alone does not reduce the worst case time complexity of $O(mn)$. In cases like this we match large portions of the pattern to the text, but the bad character rule ignores this information and only considers the character that caused the mismatch in the text. To rectify this we need to develop a second shift rule - the *good suffix rule* - that considers how to shift the pattern based on the region that **matched** the text. The bad character and the good suffix rules are then used in conjunction determine the best possible shift for a given alignment.

Let's develop the good suffix rule by considering an extended example and relating it to the general case as we proceed. Consider the pattern $\mathbf{pat}[1\ldots11] = acababacaba$ and it's alignment with a substring $\mathbf{txt}[j\ldots j+10]$ of a reference text shown in figure 7.

		j	j+1	j+2	j+3	j+4	j+5	j+6	j+7	j+8	j+9	j+10	j+11	j+12	j+13	j+14	j+15	
txt	…	**a**	**c**	**x**	**c**	**c**	**y**	**x**	**b**	**a**	**b**	**a**	**c**	**a**	**b**	**a**	**a**	…
		1	2	3	4	5	6	7	8	9	10	11						
pat		**a**	**c**	**a**	**b**	**a**	**b**	**a**	**c**	**a**	**b**	**a**						

Figure 7: An alignment between $\mathbf{txt}[j\ldots j+10]$ and the pattern $\mathbf{pat}[1\ldots11] = acababacaba$.

If we make a right-to-left scan we discover that,

$$\mathbf{pat}[9\ldots11] = \mathbf{txt}[j+8\ldots j+10] \text{ and } \mathbf{pat}[8] \neq \mathbf{txt}[j+7],$$

as indicated by the green and red boxes, respectively in figure 8.

		j	j+1	j+2	j+3	j+4	j+5	j+6	j+7	j+8	j+9	j+10	j+11	j+12	j+13	j+14	j+15	
txt	…	**a**	**c**	**x**	**c**	**c**	**y**	**x**	**b**	**a**	**b**	**a**	**c**	**a**	**b**	**a**	**a**	…
		1	2	3	4	5	6	7	8	9	10	11						
pat		**a**	**c**	**a**	**b**	**a**	**b**	**a**	**c**	**a**	**b**	**a**						

Figure 8: The matching regions and mismatched characters are indicated by the green and red boxes respectively.

In this alignment the bad character rule would suggest shifting the pattern right by 1 place as $R(b) = 10$, while the extended bad character rule would suggest a shift of 2 places as $R_8(b) = 6$. However, we can do better than this.

Let's first generalise this situation and imagine that during a particular iteration we've aligned $\mathbf{pat}[1\ldots m]$ with $\mathbf{txt}[j\ldots j+m-1]$. In addition, assume that,

$$\mathbf{pat}[k+1\ldots m] = \mathbf{txt}[j+k\ldots j+m-1] \text{ and } \mathbf{pat}[k] \neq \mathbf{txt}[j+k-1],$$

in complete analogy to our example as shown in figure 9, where we've let x and y denote $\mathbf{txt}[j+k-1]$ and $\mathbf{pat}[k]$ respectively.

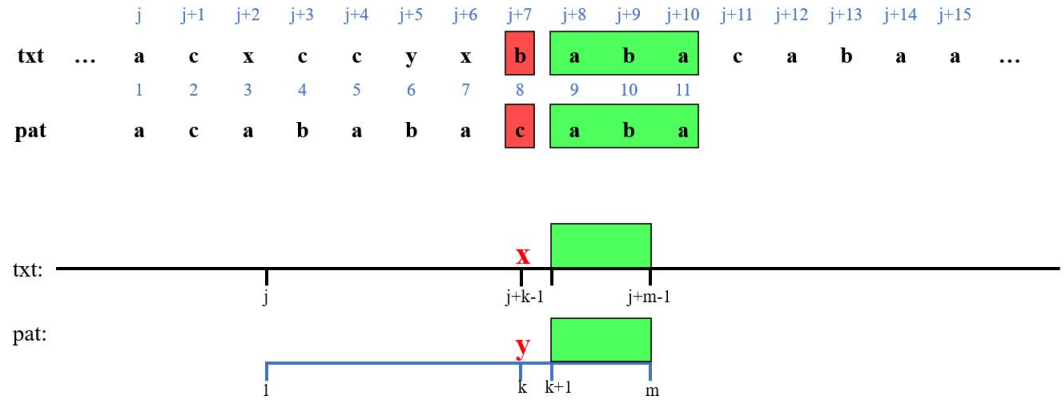


Figure 9: A general alignment between $\mathbf{pat}[1 \ldots m]$ and $\mathbf{txt}[j \ldots j+m-1]$. The regions $\mathbf{pat}[k+1 \ldots m]$ and $\mathbf{txt}[j+k \ldots j+m-1]$ are found to match via right-to-left scan, before the characters $\mathbf{pat}[k] \equiv y$ and $\mathbf{txt}[j+k-1] \equiv x$ are found to mismatch.

Now assume that pattern contains a substring (excluding the suffix itself) that matches the suffix $\mathbf{pat}[k+1 \ldots m]$ that matched the text, as shown in figure 10. If we let p denote the right endpoint of this copy of the *good suffix*, then shifting the pattern to the right by $m-p$ positions will align it with the matched region in the text, as shown in figure 11. Our good suffix shift is guaranteed to be safe provided that we use the **rightmost** substring that matches the suffix. However, we can impose an additional constraint on this substring to obtain more useful shifts. In particular, we require that the character (z) immediately preceding the substring (see figure 10) to be different to $\mathbf{pat}[k] \equiv y$, otherwise we would mismatch in the exact same manner after the shift as shown in figure 11. This variant of the good suffix rule is called the *strong good suffix* rule and we will simply refer to it as the good suffix rule from this point onwards.

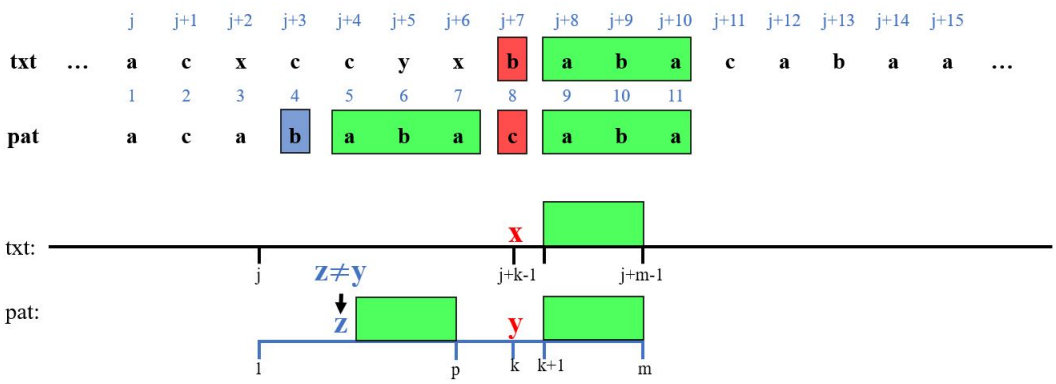


Figure 10: The pattern contains a substring that matches the suffix $\mathbf{pat}[k+1 \ldots m]$ that matched the text. The substring (denoted by the leftmost green box) that matches its suffix (the rightmost green box in the pattern) ends at position p, and the character immediately preceding this substring (z) is different to the mismatched character $\mathbf{pat}[k] \equiv y$.

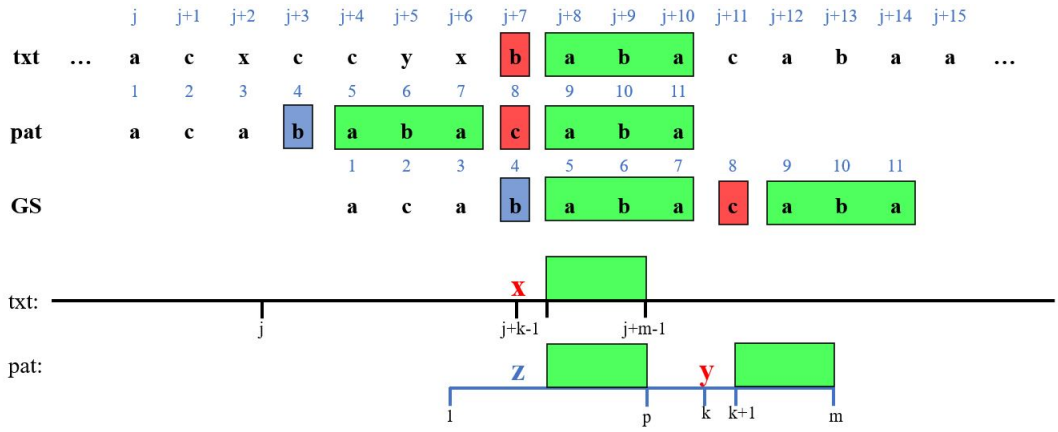


Figure 11: Shifting the pattern to the right by $m - p$ positions will align the substring that matches its suffix with the region that was originally matched in the text.

To enable good suffix shifts we need to store the rightmost substrings that match their suffix. In particular, for each suffix in the pattern, we want to store the right end point p of the rightmost substring that matches the suffix and is immediately preceded by a character that differs from the character that immediately precedes the suffix. To formalise this idea we will make use of the following definition.

Definition 1: Z^{suffix}-values

Given $\mathbf{pat}[1 \ldots m]$ define Z_i^{suffix}, for $i < m$, as the length of the longest substring ending at position i of **pat** that matches its suffix, i.e. $\mathbf{pat}[i - Z_i^{\text{suffix}} + 1 \ldots i] = \mathbf{pat}[m - Z_i^{\text{suffix}} + 1 \ldots m]$.

Now for each suffix starting at position j ($\mathbf{pat}[j \ldots m]$) in the pattern, define,

$$gs(j) = p,$$

where p is the rightmost position such that,

$$\mathbf{pat}[p - Z_p^{\text{suffix}} + 1 \ldots p] = \mathbf{pat}[j \ldots m] \text{ and } \mathbf{pat}[p - Z_p^{\text{suffix}}] \neq \mathbf{pat}[j - 1].$$

Example 6: Good suffix values

To visualise the above definitions lets consider: $\mathbf{pat}[1 \ldots 11] =$ 1 2 3 4 5 6 7 8 9 10 11 a c a b a b a c a b a the corresponding Z^{suffix} and good suffix values are:

i	1	2	3	4	5	6	7	8	9	10	11	12
pat	a	c	a	b	a	b	a	c	a	b	a	
Z_i^{suffix}	1	0	1	0	5	0	3	0	1	0	-	-
$gs(i)$	0	0	0	0	0	0	5	0	7	0	9	10

Despite their verbose definition the Z^{suffix}-values are relatively simple. For instance, $Z_1^{\text{suffix}} = 1$ as the longest substring ending at position 1 that matches a suffix of the pattern is $\mathbf{pat}[1] = a$, which happens to be the only substring that ends at position 1, and it is of length 1. Similarly, $Z_7^{\text{suffix}} = 3$ as the longest substring that ends at position 7, and matches its suffix is $\mathbf{pat}[5 \ldots 7] = aba$, which has length 3 and matches the suffix $\mathbf{pat}[9 \ldots 11] = aba$. Note that the Z^{suffix}-values are not defined for $i \geq 11$. The good suffix values are similarly straightforward.

For example, $gs(7) = 5$ because 5 is the right end point, of the rightmost substring that matches the suffix $\mathbf{pat}[7 \ldots 11]$ and isn't preceded by a 'b' ($\mathbf{pat}[6] \equiv b$). That is, we have $\mathbf{pat}[1 \ldots 5] = \mathbf{pat}[7 \ldots 11]$ and the substring $\mathbf{pat}[1 \ldots 5]$ is a prefix and so isn't preceded by any character. To relate this to the Z^{suffix}-values,

note that,

$$\begin{aligned}\mathbf{pat}[p - Z_p^{\text{suffix}} + 1 \ldots p] &= \mathbf{pat}[m - Z_p^{\text{suffix}} + 1 \ldots m] \text{ (by definition of } Z_p^{\text{suffix}}) \\ \mathbf{pat}[5 - 5 + 1 \ldots 5] &= \mathbf{pat}[11 - 5 + 1 \ldots 11] \ (p = 5,\ Z_5^{\text{suffix}} = 5, \text{ and } m = 11) \\ \mathbf{pat}[1 \ldots 5] &= \mathbf{pat}[7 \ldots 11],\end{aligned}$$

as Z_5^{suffix} denotes the longest substring ending at position 5 with these properties we are guaranteed that,

$$\begin{aligned}\mathbf{pat}[5 - Z_5^{\text{suffix}}] &\neq \mathbf{pat}[7 - 1] \text{ (j=7 in this case)} \\ \mathbf{pat}[0] &\neq \mathbf{pat}[6],\end{aligned}$$

where here $\mathbf{pat}[0]$ doesn't exist and so matches nothing.

Similarly, $gs(9) = 7$ as $\mathbf{pat}[5 \ldots 7] = \mathbf{pat}[9 \ldots 11]$ and $\mathbf{pat}[4] \equiv b \neq \mathbf{pat}[8] \equiv c$. Finally, we define $gs(12) = 10$ as the suffix $\mathbf{pat}[12 \ldots 11]$ we are trying to match is the empty string and so we only require the rightmost position p where $\mathbf{pat}[p] \neq \mathbf{pat}[11] \equiv a$. The first such position is position 10 as $\mathbf{pat}[10] \equiv b$.

Now that we've defined our good suffix values we're finally ready to see how the Z-algorithm plays a fundamental role in the preprocessing phase of Boyer-Moore.

Preprocessing the good suffix values

Given a pattern $\mathbf{pat}[1 \ldots m]$, to preprocess the good suffix values ($\{gs(i)\}$ for $1 \leq i \leq m + 1$) in $O(m)$-time we need to compute the Z^{suffix}-values for each position $1 \leq i < m$ in **pat**. Recall that Z_i denotes the length of the longest substring, that starts at position i and matches its prefix. If we were to reverse the pattern and run the Z-algorithm, then Z_i would now denote the longest substring that ended at i (in the original pattern) and matched its **suffix**. But this is precisely Z_i^{suffix}! Thus, we can compute the Z^{suffix}-values in $O(m)$-time by running the Z-algorithm on reverse(**pat**). The Z^{suffix}-values can then be mapped to the corresponding good suffix values using the pseudocode below.

```
m := |pat|
for j from 1 to m+1  do
    goodsuffix(j) := 0
endfor

for p from 1 to m-1  do
    j := m - Z^{suffix}_p + 1
    goodsuffix(j) := p
endfor
```

After running the Z-algorithm on reverse(**pat**) the Z^{suffix}-values output will refer to the reversed pattern. Due to this we need to reverse the resulting Z-value array so that the Z^{suffix}-values refer to the unreversed pattern

Finally, let's return to the general alignment between the pattern and the text as shown in figure 10 and determine how to use the good suffix rule during the search phase. If a mismatch occurs at position k in the pattern and $gs(k + 1) > 0$ then it is safe to shift by $m - gs(k + 1)$ places. If $gs(k + 1) = 0$ then this indicates that there's no satisfactory substring that we can align with the matched region in the text. However, if we shift by m places we could miss an occurrence of the pattern as there may be some suffix of the good suffix that occurs elsewhere in the pattern. In particular, we are concerned with the case where a suffix of the good suffix matches the prefix of the pattern (why we are only concerned with prefix matches will be explored in tutorials), as shown in figure 12. The longest such prefix will give the largest safe shift and we call this the *matched prefix*.

Definition 2: Matched prefix

The matched prefix value $mp(k + 1)$, associated with the suffix $\mathbf{pat}[k + 1 \ldots m]$ is defined to be the largest suffix of $\mathbf{pat}[k + 1 \ldots m]$ that matches the prefix $\mathbf{pat}[1 \ldots m - k]$.

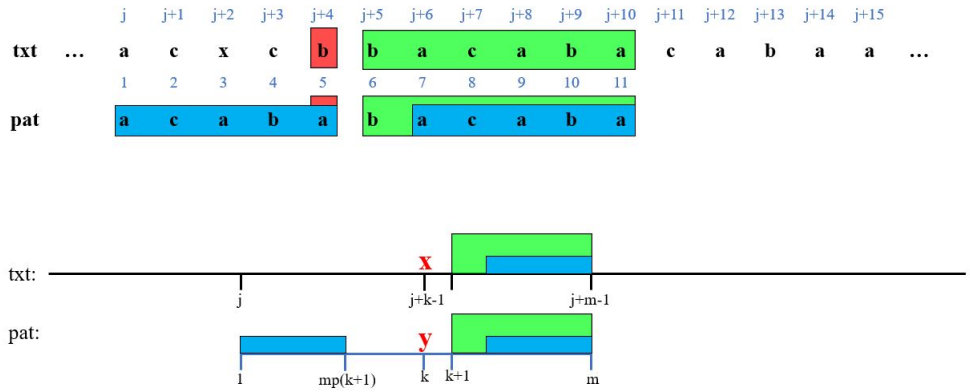


Figure 12: In the case where the good suffix value that corresponds to the matched region of the pattern ($\mathbf{pat}[k+1 \ldots m]$) is 0 ($gs(k+1) = 0$) we define the matched prefix value $mp(k+1)$, to denote the length of the longest prefix $\mathbf{pat}[1 \ldots m-k]$ that matches a suffix of $\mathbf{pat}[k+1 \ldots m]$. Here the matched region of the pattern and text are shown in green, while the longest prefix that matches a suffix of this region is shown in blue.

Example 7: Matched prefix values

To see how the matched prefix values compare to Z^{suffix} and good suffix values once again consider: $\mathbf{pat}[1 \ldots 11] = \overset{1}{\mathsf{a}}\ \overset{2}{\mathsf{c}}\ \overset{3}{\mathsf{a}}\ \overset{4}{\mathsf{b}}\ \overset{5}{\mathsf{a}}\ \overset{6}{\mathsf{b}}\ \overset{7}{\mathsf{a}}\ \overset{8}{\mathsf{c}}\ \overset{9}{\mathsf{a}}\ \overset{10}{\mathsf{b}}\ \overset{11}{\mathsf{a}}$. The corresponding matched prefix values are:

i	1	2	3	4	5	6	7	8	9	10	11	12
pat	a	c	a	b	a	b	a	c	a	b	a	
Z_i^{suffix}	1	0	1	0	5	0	3	0	1	0	-	-
$gs(i)$	0	0	0	0	0	0	5	0	7	0	9	10
$mp(i)$	11	5	5	5	5	5	5	1	1	1	1	0

The matched prefix values are somewhat easier to 'eyeball' than the good suffix values. For example, $mp(9) = 1$ as the longest suffix of $\mathbf{pat}[9 \ldots 11] \equiv aba$ that matches its prefix $\mathbf{pat}[1 \ldots 3] = aca$ is 'a', which is of length 1. Similarly, $mp(5) = 5$ as the longest suffix of $\mathbf{pat}[5 \ldots 11] = abacaba$ that matches the prefix $\mathbf{pat}[1 \ldots 7] = acababa$ is 'acaba', which has length 5. Finally, $mp(12) = 0$ as the suffix $\mathbf{pat}[12 \ldots 11]$ is the empty string and the longest suffix of the empty string that matches the prefix has length 0. The practical importance of this value is as follows: $mp(12)$ is only used if $gs(12) = 0$, however $gs(12) = 0$ implies that there is no character in the pattern that is different to $\mathbf{pat}[11] \equiv a$, that is the pattern is all a's. In this case, if we mismatch on $\mathbf{pat}[11]$ (so that we try to shift using $gs(12)$) there is no character in the pattern that won't mismatch in the same way that $\mathbf{pat}[11]$ did. So we shift the whole pattern past the mismatched character in the text, which is achieved by shifting by $m - mp(12) = m$ positions.

We can use the matched prefix values to shift the pattern in an identical manner to the good suffix values. If we mismatch at position k and $gs(k+1) = 0$ then we shift by $m - mp(k+1)$ places, as shown in figure 13. This aligns the prefix that matched part of the good suffix to the corresponding matched region in the text. Finally, if we find a complete match we shift the pattern by $m - mp(2)$ places and this will be explained in tutorials.

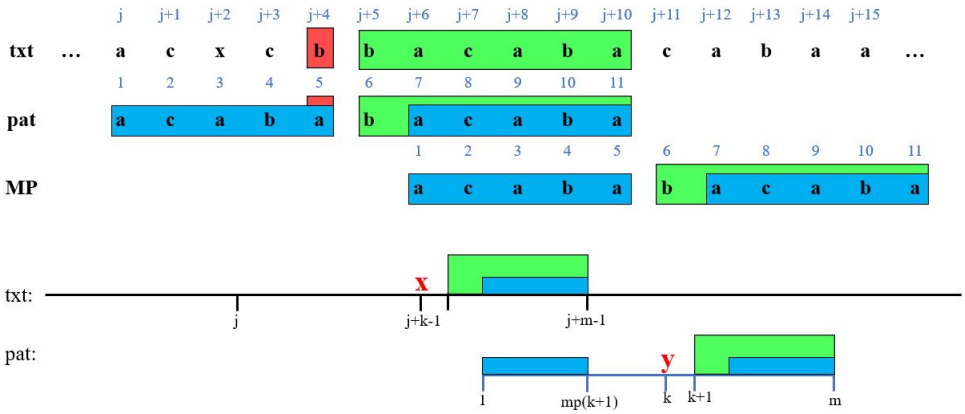


Figure 13: After a mismatch occurs at position k in the pattern, we shift the pattern to the right by $m - mp(k+1)$ places to align the matched prefix with the corresponding region that was matched in the text.

Question 4: Preprocessing matched prefix values

Given $\mathbf{pat}[1 \dots m]$ describe using pseudocode, or otherwise, how the matched prefix values can be obtained in $O(m)$-time by preprocessing the pattern. *Hint you should make use of the Z-algorithm.*

Question 5: Good suffix vs bad character

Return to example 5 and determine the alignments explored by the good suffix rule. How does this compare to the bad character rule?

1.1.5 Galil's optimisation

Boyer-Moore uses both the (extended) bad character and good suffix rules to try and determine the best shift between alignments. In particular, since both rules guarantee that their shift is safe, Boyer-Moore will choose the largest of the two suggested shifts. However, this still doesn't yield a linear time method as the next example demonstrates.

Example 8: Worst case of the good suffix rule

Let:

$\mathbf{txt}[1 \dots 10] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{a}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{a}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\mathtt{a}}\ \overset{8}{\mathtt{a}}\ \overset{9}{\mathtt{a}}\ \overset{10}{\mathtt{a}}$ and $\mathbf{pat}[1 \dots 3] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{a}}\ \overset{3}{\mathtt{a}}$.
In this case we find a complete match in every valid alignment between **pat** and **txt**. Obviously, we need to explore every alignment, and Boyer-Moore will do exactly that (you should confirm that $m - mp(2) = 1$ for this pattern). However, if we explore all alignments and find a complete match in each one, then clearly we still have an $O(mn)$-time method.

Example 8 illustrates that even with the good suffix rule there are still cases for which Boyer-Moore isn't a linear time method. To rectify this, we introduce one further optimisation known as Galil's optimisation that will guarantee Boyer-Moore runs in $O(m+n)$-time in the worst case.

Consider some general iteration where $\mathbf{pat}[1 \dots m]$ is being compared with $\mathbf{txt}[j \dots j+m-1]$ via a right-to-left scan. Assume that the pattern matches the text down to some position $k+1$, i.e.

$$\mathbf{pat}[k+1 \dots m] = \mathbf{txt}[j+k \dots j+m-1],$$

and the iteration is then ended due to a mismatch at position k, or because an occurrence of the pattern was found. The pattern is then shifted right in accordance with either the bad character rule, or the good suffix rule. If after

this shift, the left end of the pattern ($\mathbf{pat}[1]$) lies within (excluding the left endpoint) the region that was previously matched in the text ($\mathbf{txt}[j+k \ldots j+m-1]$), then the prefix of the pattern ($\mathbf{pat}[1 \ldots]$) must match this section of the text up until $\mathbf{txt}[j+m-1]$ (see figure 14). Therefore, we can avoid comparing the characters of this prefix in the next iteration and the right-to-left scan can be terminated early if position $\mathbf{txt}[j+m-1]$ is reached. In this case we can conclude that an occurrence of the pattern has been found in the text without making further comparisons.

To better rationalise Galil's optimisation, let's consider what kind of shifts could produce such an alignment. It cannot be good suffix shift using $gs(k+1)$ unless the matching substring also happens to be a prefix i.e. $gs(k+1) = mp(k+1)$. This is because any other good suffix shift wouldn't move the left end of the pattern past the bad character $\mathbf{txt}[j+k-1] \equiv x$. If this is the case, the good suffix shift is equivalent to the matched prefix shift and we are guaranteed that the prefix of the pattern matches the previously matched region in the text. Similarly, it cannot be a bad character shift unless the character x doesn't occur in the pattern, or to the left of k in the case of the extended bad character rule. This is because if x did occur in a suitable position in the pattern, this occurrence would be aligned with the mismatched x in the text. If an appropriate occurrence of x doesn't occur in the pattern then the bad character rule would align $\mathbf{pat}[1]$ with $\mathbf{txt}[j+k]$. In this case we cannot be sure that the prefix matches the corresponding region in the text. Thus, we can't apply Galil's optimisation when $\mathbf{pat}[1]$ is aligned with $\mathbf{txt}[j+k]$ as it may be due to a matched prefix shift or a bad character shift. If $\mathbf{pat}[1]$ is aligned with any character $\mathbf{txt}[j+k+1 \ldots j+m-1]$ then we can be sure that it was a matched prefix shift and the prefix is guaranteed to match the corresponding region in the text after the shift.

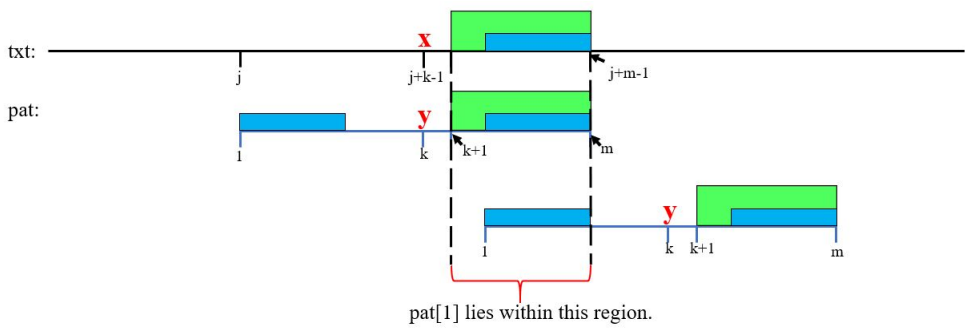


Figure 14: Depiction of Galil's optimisation. If in the next iteration, the left end of the pattern lies within (excluding the left end point $\mathbf{txt}[j+k]$) the region of the text that was matched to the pattern in the previous iteration (the green region in $\mathbf{txt}$), then the prefix of the pattern $\mathbf{pat}[1 \ldots]$ must match the text up until $\mathbf{txt}[j+m-1]$.

In light of the previous discussion we can phrase Galil's optimisation far more succinctly. Galil's optimisation simply says that if we shift the pattern using $mp(k+1)$ (or $gs(k+1) = mp(k+1)$), then we can avoid performing explicit comparisons on the prefix $\mathbf{pat}[1 \ldots mp(k+1)]$ in the next iteration. With Galil's optimisation Boyer-Moore is guaranteed to be an $O(m+n)$-time algorithm in the worst case. However, one might wonder why we restrict such an optimisation to matched prefix shifts, since we know that after **any** good suffix shift there will be a region of the pattern that is guaranteed to match the text. If we return to figure 11, we can see that in the case where $gs(k+1) > 0$ the green region $\mathbf{pat}[p-m+k+1 \ldots p]$ matches the previously matched region in the text i.e. ,

$$\mathbf{pat}[p-m+k+1 \ldots p] = \mathbf{txt}[j+k \ldots j+m-1].$$

So let's extend Galil's optimisation to this case.

Galil's optimisation

Consider some general iteration where $\mathbf{pat}[1 \ldots m]$ is being compared with $\mathbf{txt}[j \ldots j+m-1]$ via a right-to-left scan. Assume that the pattern matches the text down to some position $k+1$, i.e.

$$\mathbf{pat}[k+1 \ldots m] = \mathbf{txt}[j+k \ldots j+m-1],$$

and the iteration is then ended due to a mismatch at position k, or because an occurrence of the pattern was found. Assume the pattern is shifted to the right with respect to the text using the good suffix rule and that $gs(k+1) = p$.

- If $p > 0$, then,

$$\mathbf{pat}[p - m + k + 1 \ldots p] = \mathbf{txt}[j + k \ldots j + m - 1].$$

and we can skip making comparisons in this region in the right-to-left scan in the next iteration as shown in figure 15.

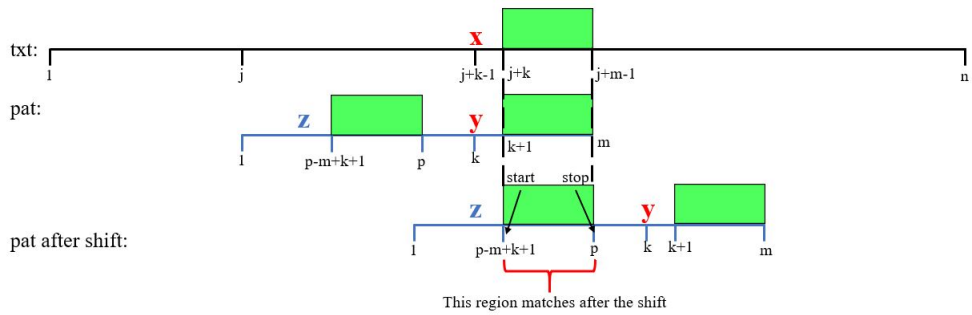


Figure 15: After making a good suffix shift by $m - gs(k + 1)$ places, where $gs(k + 1) = p > 0$, the regions $\mathbf{pat}[p - m + k + 1 \ldots p]$ and $\mathbf{txt}[j + k \ldots j + m - 1]$ are known to match. The comparisons in these regions can be skipped in the subsequent right-to-left scan.

- If $p = 0$ then we made a matched prefix shift $(m - mp(k + 1))$ and we know that,

$$\mathbf{pat}[1 \ldots mp(k + 1)] = \mathbf{txt}[j + m - mp(k + 1) \ldots j + m - 1],$$

and in the next iteration we can terminate our right-to-left scan prematurely if it reaches $\mathbf{txt}[j + m - 1]$ and conclude that an occurrence of the pattern has been found, as shown in figure 16.

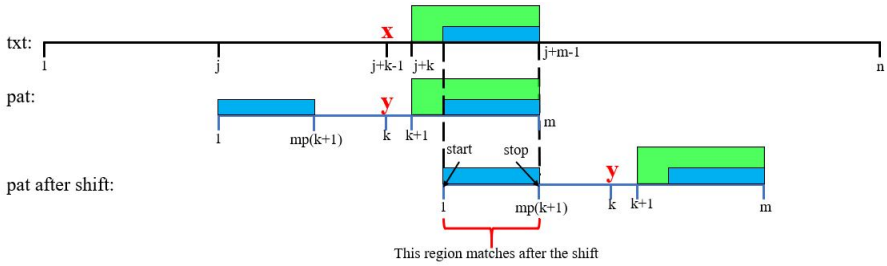


Figure 16: After making a matched prefix shift by $m - mp(k + 1)$ places, the regions $\mathbf{pat}[1 \ldots mp(k + 1)]$ and $\mathbf{txt}[j + m - mp(k + 1) \ldots j + m - 1]$ are known to match. The comparisons in these regions can be skipped in the subsequent right-to-left scan.

It is understandable if the above seem like two separate optimisations, but intuitively they both just tell us to avoid making comparisons in regions of the text that we already know to match the pattern. Furthermore, they can be implemented as a single optimisation by augmenting the search phase of the algorithm with two pointers into each alignment. Let the first called *stop*, point to the rightmost position in the pattern (or the text) that we already know to match the pattern. Similarly, the let the second called *start*, point to the leftmost position in the pattern (or the text) that we already know to match the pattern.

The idea is to use *stop* and *start* (or equivalent variables) to stop and start the right-to-left scan in order to skip over regions of the pattern that we already know match the text. For instance, if $gs(k + 1) = p > 0$ and we shift the pattern according to the good suffix rule we would set:

- $stop = p$,
- $start = p - m + k + 1$.

Comparisons begin with a right-to-left pass through $\mathbf{pat}[stop+1 \ldots m]$ and then halt once $\mathbf{pat}[stop]$ is reached. The comparisons are then restarted at $\mathbf{pat}[start-1]$ and continue right-to-left through $\mathbf{pat}[1 \ldots start-1]$. This means that comparisons are never performed for characters in the region $\mathbf{pat}[start \ldots stop]$. We can use the same method if $gs(k+1) = 0$ and we make a matched prefix shift. In this case we could set,

- $stop = mp(k+1)$,
- $start = 1$,

where the fact that $start - 1 = 0$ denotes that comparisons don't need to be restarted once $\mathbf{pat}[stop]$ is reached, although many other equivalent implementations exist.

1.1.6 Summary

Boyer-Moore preprocessing

The following tasks are performed by preprocessing the pattern and assuming a fixed size alphabet and input $\mathbf{pat}[1 \ldots m]$, can be completed in $O(m)$ time:

- Preprocess the $R(x)$ ($R_k(x)$) values for the bad character (extended bad character) rule.
- Preprocess the gs and mp values needed for the good suffix rule.

Boyer-Moore search phase

Given inputs $\mathbf{pat}[1 \ldots m]$ and $\mathbf{txt}[1 \ldots m]$ begin by aligning $\mathbf{pat}[1 \ldots m]$ with $\mathbf{txt}[1 \ldots m]$. The algorithm proceeds as follows:

1. Compare the current alignment in a right-to-left scan, applying Galil's optimisation to terminate the scan prematurely if appropriate.
2. After the scan has terminated due to a mismatch, or an occurrence of the pattern being found (which is output appropriately):
   (a) Use the (extended) bad character rule to determine how many places to the right **pat** should be shifted under **txt**. Call this amount $n_{\text{badcharacter}}$.
   (b) Use the good suffix and the matched prefix rules to determine how many places to the right **pat** should be shifted under **txt**. Call this amount $n_{\text{goodsuffix}}$.
3. Shift **pat** to the right under **txt** by $max(n_{\text{badcharacter}}, n_{\text{goodsuffix}})$ places. Return to step 1.

Time complexity of the Boyer-Moore algorithm

Although the details are not examinable we would be remiss to not at least acknowledge that when used in string matching applications that involve natural languages the search phase of Boyer-Moore typically exhibits a sublinear time complexity of $O\left(\frac{n}{m}\right)$. This is quite startling as it implies that we expect the algorithm to not even examine every character in the text while still finding all occurrences of the pattern. The worst case of Boyer-Moore with the original Galil's optimisation is provably $O(m+n)$. However, the proof is rather lengthy and we won't reproduce it here.

2 The Knuth-Morris-Pratt (KMP) algorithm

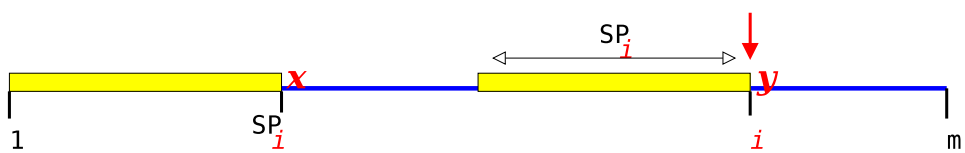
As discussed previously the KMP algorithm is inferior in performance to Boyer-Moore and is rarely used in practice, but it's popularity in undergraduate computer science courses and historical significance warrant a brief examination.

2.1 The algorithm

In a similar vein to Boyer-Moore's good suffix rule, KMP attempts to avoid unnecessary comparisons by using the information it gains from the matches that occur within an alignment. In addition, the shift rule utilised by KMP is reminiscent of the matched prefix shifts used in the good suffix rule. Just as in the matched prefix shifts, KMP seeks to align prefixes of the pattern with suffixes of the matched region in the text. However, KMP uses a left-to-right scan rather than Boyer-Moore's right-to-left scan, and so the matched regions may no longer correspond to a suffix of the pattern. The following definition will help us to more precisely define the regions of the pattern we are referring to.

Definition 3: SP_i-values

Given a pattern $\mathbf{pat}[1 \dots m]$, define SP_i for $1 \leq i \leq m$, as the length of the longest proper suffix of $\mathbf{pat}[1 \dots i]$ that matches a prefix of $\mathbf{pat}$, such that $\mathbf{pat}[i+1] \neq \mathbf{pat}[SP_i + 1]$.



Example 9: SP_i-values

Let $\mathbf{pat}=$ `b b c c a e b b c a b d` the corresponding SP_i-values are

$$\begin{array}{ccc} SP_1 = 0 & SP_2 = 1 & SP_3 = 0 \\ SP_4 = 0 & SP_5 = 0 & SP_6 = 0 \\ SP_7 = 0 & SP_8 = 1 & SP_9 = 3 \\ SP_{10} = 0 & SP_{11} = 1 & SP_{12} = 0 \end{array}$$

Preprocessing SP_i-values

The SP_i-values for a given pattern $\mathbf{pat}[1 \dots m]$ denote the length of particular substrings that match the prefix of the pattern, and so it is not surprising (in hindsight) that they can be computed using the Z-values of $\mathbf{pat}$. In particular, to preprocess the SP_i-values in $O(m)$-time we compute the Z-values of $\mathbf{pat}$ using the Z-algorithm and then use the pseudocode below determine the SP_i-values for $\mathbf{pat}$.

```
m := |pat|
for i from 1 to m  do
    SP_i := 0
endfor

for j from m down to 2  do
    i := j + Z_j - 1
    SP_i := Z_j
endfor
```

The logic of KMP is similar to that of Boyer-Moore's good suffix rule. In a general iteration of the algorithm the pattern is compared to the text in a left-to-right scan. If we assume the scan terminates due to a mismatch at position $i+1$ in the pattern, as shown in figure 17, then the algorithm will seek to align the longest prefix that matches a proper suffix of the matched region in the pattern with the corresponding characters that were matched in the text. The length of this region is given by SP_i and if we shift the pattern to the right by $i - SP_i$ places we achieve the desired alignment. In addition, the requirement that $\mathbf{pat}[i+1] \neq \mathbf{pat}[SP_i + 1]$ ensures that we don't mismatch in the same way in the next iteration. Similarly, if we find an occurrence of the pattern in the text we shift the pattern by $m - SP_m$ places. Finally, in analogy to Galil's optimisation, after a shift we know that the prefix $\mathbf{pat}[1 \ldots SP_i]$ already matches the text and so we only make explicit comparisons for $\mathbf{pat}[SP_i + 1 \ldots]$.

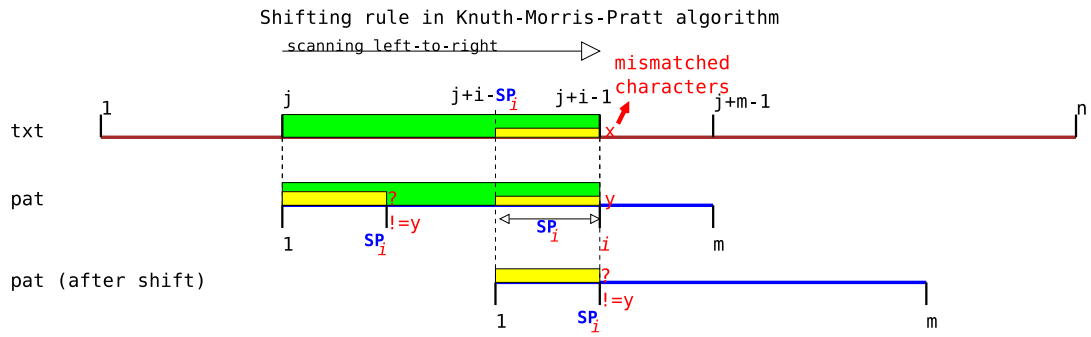


Figure 17: The KMP shift rule. In a general alignment $\mathbf{pat}[1 \ldots m]$ is compared via left-to-right scan with $\mathbf{txt}[j \ldots j+m-1]$ and the substrings $\mathbf{pat}[1 \ldots i]$ and $\mathbf{txt}[j \ldots j+i-1]$ are found to match, with the characters $\mathbf{pat}[i+1] \equiv y$ and $\mathbf{txt}[j+i]$ found to mismatch. The pattern is then shifted right by $i - SP_i$ places to align the prefix $\mathbf{pat}[1 \ldots SP_i]$ with $\mathbf{txt}[j+i-SP_i]$.

KMP Summary

Given inputs $\mathbf{pat}[1 \ldots m]$ and $\mathbf{txt}[1 \ldots n]$, begin by aligning $\mathbf{pat}[1 \ldots m]$ with $\mathbf{txt}[1 \ldots m]$. The algorithm proceeds as follows:

1. Compare the current alignment using a left-to-right scan, skipping the prefix $\mathbf{pat}[1 \ldots SP_i]$ where appropriate.
2. If the scan terminates due to a mismatch at position $i+1$ in the pattern (see figure 17), shift the pattern to the right by $i - SP_i$ places.
3. Otherwise if an occurrence of **pat** is found in **txt**, shift the pattern to the right by $m - SP_m$ places.
4. Return to step 1 and begin scanning the new alignment.

Worst case time complexity of the KMP algorithm

Given inputs $\mathbf{pat}[1 \ldots m]$ and $\mathbf{txt}[1 \ldots n]$ the KMP algorithm has worst case time complexity of $O(m+n)$.
Proof:
We have already seen that the preprocessing phase requires computing the Z-values for the pattern and mapping them to the corresponding SP_i values, which can be done in $O(m)$-time, and so all that remains is to bound the number of character comparisons the algorithm makes.

After any shift, the comparisons proceed left to right. Comparisons start with the last character compared in the previous alignment (the one that caused the mismatch), or the character to it's right if the previous alignment was a complete match. As the pattern is always shifted right, any two successive alignments share at most one comparison. We make a single comparison for every character in **txt** and in the worst case we do s of these comparisons a second time, giving $n+s$ comparisons overall, where s is the number of times the algorithm shifts the pattern. But we know that the size of each shift δ is such that $1 \leq \delta \leq m$, and so we must have $s < n$ since after we have made n shifts we will have exhausted all valid alignments between **pat** and **txt**. Thus, the number of comparisons made by the algorithm is bounded by $2n$, giving an $O(m+n)$-time algorithm in the worst case. ■